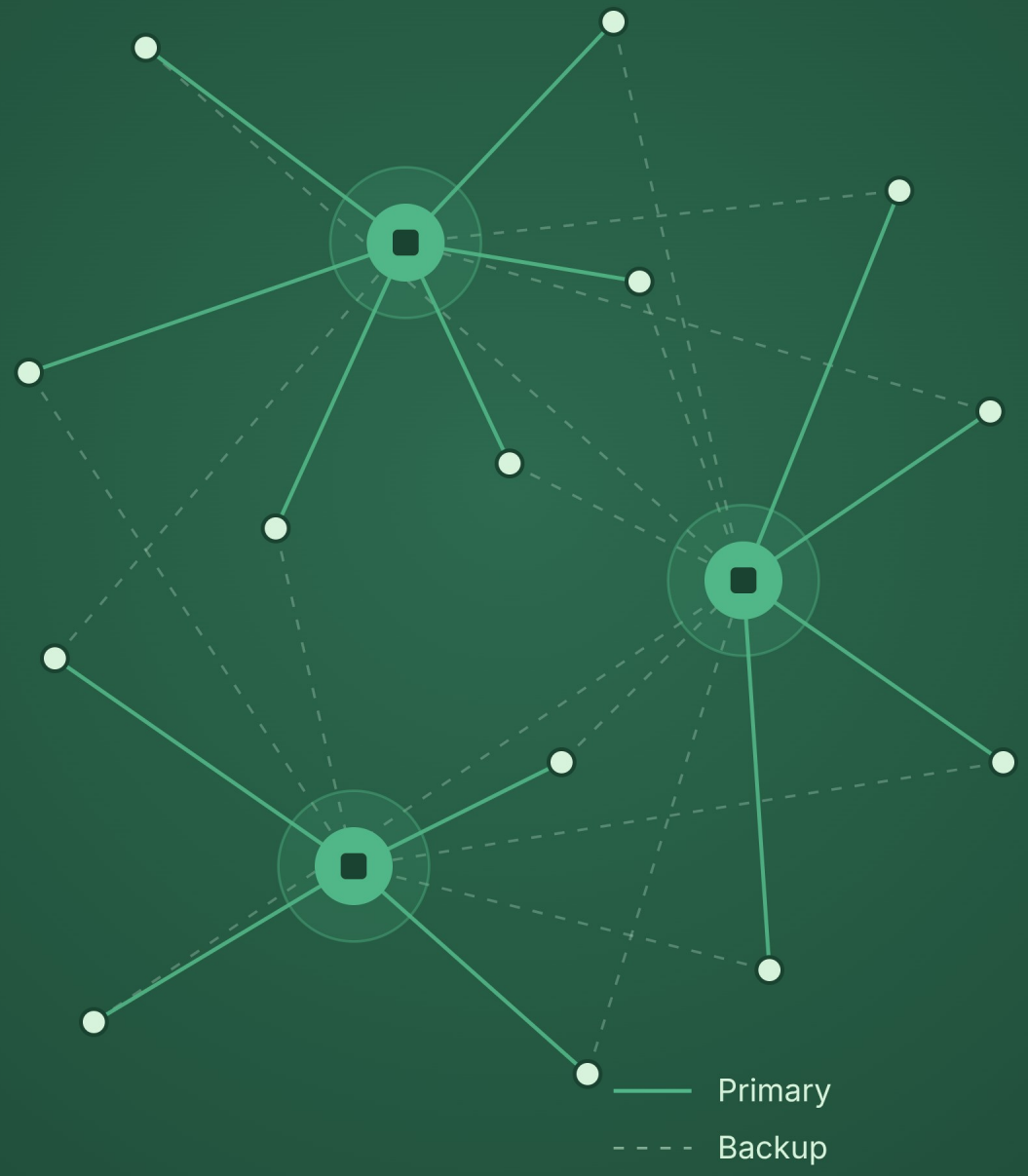
AI CONSULTANT TRACK · PROJECT PRESENTATION

Green Warehouse Network

Resilient warehouse sites that minimize CO_2 or cost, explored in plain English.

Fatma · November 12, 2026



THE PROBLEM

Warehouse sites trade off CO_2, cost and resilience

Two objectives

Minimizing CO_2 and minimizing cost can open different sites. Planners need both numbers.

Failure cover

Every customer needs a primary and a backup. If one warehouse fails, the rest must absorb its demand.

Slow what-ifs

A site closes, demand grows, trucks go electric. Every change means re-solving the network.

The tool decides how many warehouses to open, where, and which two serve each customer.

THE USER AND WHY IT MATTERS

Built for the person who has to defend the network

WHO IT IS FOR

Teams that choose warehouse sites

- Logistics companies
- Third-party logistics providers
- Product companies that organize their own shipments with a logistics provider

WHY IT MATTERS

Resilience has a price

Reports cost and CO_2 of the backup-ready network against a single center-of-gravity warehouse.

Green has a price

Shows the extra € per tonne of CO_2 saved, next to the carbon price.

Answers in a conversation

Describe a scenario in plain English, get a solved result in under 60 seconds.

AUTONOMY

From one message to a finished report

1
Trigger
Chat message or button

→

2
Parse intent
LLM, structured output

→

3
Validate
Ranges, site IDs, conflicts

→

4
Run model
MIP solved with CBC

5
Feasibility
Reason code if infeasible

→

6
Compare
Change list, failure sim

→

7
Explain
Reply of 120 words or fewer

→

8
Report
HTML file, no email

Only pause If a request is invalid or infeasible, the agent asks one specific question, then resumes.

AUTONOMY · WORKED EXAMPLE

“What if warehouse B goes down?”

Step	What happens	Tool
1 · Trigger	A chat message, or a quick-action button	Streamlit chat
2 · Parse intent	Sets close_sites = [B]; all else default	Structured output
3 · Validate	Site B exists; ranges and conflicts pass	validate_scenario
4 · Run model	Solves primary, backup and failure capacity	run_scenario
5 · Feasibility	Infeasible? Reason code and one proposed fix	None
6 · Compare	Change list vs last run and baseline	compare_scenarios, simulate_failure
7 · Explain	Reply of 120 words or fewer, numbers checked	None
8 · Save, report	Saves the scenario, builds the HTML report	src/report

AUTONOMY · GUARDRAILS

Autonomous, with guardrails

The solver owns every number

The LLM never writes a figure. A reply with a number not in the tool output is rejected, then regenerated once.

A closed schema

Requests become a validated Scenario. Unknown fields are impossible; out-of-range values are rejected.

Out of scope stays out

Routing and inventory requests are declined with a plain explanation. The model does not run.

One allowed pause

Invalid or infeasible? One specific question; “yes” applies the fix and re-runs.

THE REPORT

One HTML report per run

Report for the current run

Your request
The plain-English message

What changed
Parameters, KPIs, sites, customers moved

KPI cards
CO_2, cost, sites, resilience

k curve
Only if a k-sweep was run

Network map
Sites, primary and backup links, as an image

Baseline delta
Against one center-of-gravity site

Assumptions
What the numbers rest on

Map credit
OpenRouteService and OpenStreetMap attribution

Every number comes from the solver output. Built by the Download report button, or by asking “send me a report”.

THE REPORT

Why the report is useful

Price of resilience

Cost and CO_2 of the backup-ready network minus the single-warehouse baseline.

Cost of avoiding CO_2

Extra € per tonne saved, shown against the carbon price, only when both solves are optimal.

Worst-case failure

The site whose loss adds the most CO_2, and its capacity utilization.

A trail you can trust

Solver status and gap on every result; the same scenario always returns the same answer.

STACK RATIONALE

Why LangGraph, and where n8n fits

Chosen now

LangGraph

Stateful chat: checkpointer and interrupt

Typed state shared with the sidebar

Loops: ask, apply the fix, re-run

Plain Python tools call PuLP directly

Later, if needed

n8n

Built for triggers and delivery

Email, API and n8n are out of scope

Revisit for scheduled runs and email

The agent is a stateful conversation, so it runs on LangGraph.

STACK RATIONALE

The stack at a glance

Layer	Tools	Role
Data	pandas	Load and validate inputs
Distances	OpenRouteService Matrix API	Cached road km
Emissions	Climatiq API	Cached CO_2 per tonne-km
Optimization	PuLP 2.9.0 with CBC, NumPy	The MIP, center of gravity
Agent	LangGraph, LangChain, OpenAI, Pydantic	Parse, validate, route, explain
Tools	Six plain Python functions	The only path to the solver
Interface	Streamlit, Plotly, Folium	Chat, KPIs, maps, tables
Report	Python HTML template	Downloadable report

GTM SPRINT OVERVIEW · PROPOSED

Next three sprints

SPRINT 1

Real numbers

Swap placeholder costs and CO_2 for sourced figures

Pass the phase 4 acceptance checks

Rehearse the six demo scenarios

Outcome: results shown as findings

SPRINT 2

Real users

Recruit a few planners [number to confirm]

They run their own what-ifs and keep the reports

Log every question the agent should not ask

Outcome: proof the report drives decisions

SPRINT 3

Real offer

Turn pilot reports into a case study

Shape and price a consulting offer [pricing to decide]

Decide if scheduled runs and email (n8n) enter v2

Outcome: a repeatable offer and a v2 decision